Data Governance Copilot — Project Context

> Paste this at the start of any new chat to continue development instantly.

Project Goal

Build an **Agentic AI Data Governance & Analytics Assistant** — a multi-agent system where a Supervisor Agent orchestrates specialized agents to answer business users' data questions via Streamlit UI / Microsoft Teams / FastAPI. Also exposed as a **standalone MCP Server** for use inside Claude Desktop and other MCP clients.

Tech Stack (Final — Day 20)

Layer	Technology
Orchestration	LangGraph (StateGraph, sequential routing)
LLM	Groq (llama-3.3-70b-versatile) via LiteLLM fallback chain
Structured Output	Pydantic V2 + `.with_structured_output()`
RAG / Vector Store	pgvector on Neon — `NullVectorService` mock in dev
Document Loaders	LangChain (PDF, DOCX, PPTX, Excel, text)
Structured Data	Databricks SQL / SQL Warehouse (mock in dev)
Governance	Collibra REST API (USE_MCP=false in dev; MCP-routable in prod)
Ticketing	Jira REST API
Memory	MemorySaver (dev) → SqliteSaver → PostgresSaver (prod) via fallback chain
Cache	Redis (shared across ECS tasks) + in-memory fallback
Observability	LangSmith tracing
UI	Streamlit (polished, HITL panel included)
API	FastAPI + SSE + slowapi rate limiting
MCP Server	`src/mcp_server/server.py` — stdio (Claude Desktop) or SSE transport
Teams Bot	Adaptive Cards webhook + HMAC verification

Layer	Technology
Package Manager	uv
DB (primary)	**Neon PostgreSQL** — `DATABASE_URL` drives all connections (pooled pgBouncer URL)
Ingestion Pipeline	Apache Airflow 2.9/2.10 (Compose services, Neon metadata DB)
Ingestion Storage	S3 (prod) / local `docs/` folder (dev)
Infra	Docker + ECS Fargate
CI/CD	GitHub Actions
Testing	pytest (84% coverage, 240 passed)
Secrets	AWS Secrets Manager (prod)

Folder Structure

```
data-governance-copilot/
├── src/
│   ├── graph/
│   │   ├── __init__.py
│   │   ├── state.py          ✅ AgentState TypedDict + initial_state() factory
│   │   ├── intent.py         ✅ Groq structured output + LiteLLM + keyword
fallback
│   │   ├── routing.py        ✅ INTENT_AGENT_MAP + get_agents_for_intent()
│   │   ├── nodes.py          ✅ All nodes + retry + HITL + @cached_node +
synthesizer
│   │   └── graph.py          ✅ StateGraph + sequential routing + _wrap_agent
tracker
│   │                            _graph singleton, get_graph(), copilot_graph
proxy
│   ├── memory/
│   │   ├── __init__.py
│   │   └── checkpointer.py   ✅ Fallback chain: PostgresSaver → SqliteSaver →
MemorySaver
│   │                            Neon-compatible: reads DATABASE_URL or POSTGRES_*
parts
│   ├── services/             ✅ Service abstraction layer
│   │   ├── __init__.py
│   │   ├── base.py           ✅ 4 runtime_checkable Protocols
│   │   ├── factory.py        ✅ Single switching point — reads ENABLE_MOCK +
graceful fallback
```

```
|   |   ├── databricks/        ✅ real.py (DatabricksService) + mock.py
(MockDatabricksService)
|   |   ├── jira/              ✅ real.py (JiraService REST v3) + mock.py (in-
memory store)
|   |   ├── collibra/          ✅ real.py (CollibraService REST) + mock.py (canned
assets + DQ)
|   |   └── pgvector/          ✅ real.py (PGVectorService) + mock.py
(NullVectorService)
|   ├── agents/
|   |   ├── __init__.py
|   |   ├── information_agent.py    ✅ Delegates to IDataService
|   |   ├── knowledge_agent.py      ✅ Delegates to IVectorService
|   |   ├── metadata_agent.py       ✅ Delegates to IMetadataService
|   |   ├── capacity_agent.py       ✅ Delegates to ITicketService
|   |   ├── rule_agent.py           ✅ Rule registry CRUD + evaluate
|   |   └── supervisor_agent.py     ✅ Legacy stub (LangGraph nodes supersede)
|   ├── core/
|   |   ├── __init__.py
|   |   ├── base_agent.py      ✅ AgentRequest, AgentResult, BaseAgent contract
|   |   ├── logging_utils.py   ✅ Structured JSON logging, decorators, error
hierarchy
|   |   ├── guardrails.py      ✅ Length, SQL injection, prompt injection, PII
redaction
|   |   ├── cache.py           ✅ Redis + in-memory fallback + @cached_node +
invalidate_pattern()
|   |   ├── llm_factory.py     ✅ LiteLLM factory + fallback chain + _MockLLM
|   |   ├── retry.py           ✅ @with_retry + retry_agent_call()
|   |   ├── llm_guard.py       ✅ Daily token budget hard stop + get_daily_usage()
|   |   ├── vector_store.py    ✅ Backwards-compat shim → delegates to
services/pgvector/
|   |   └── mcp_client.py      ✅ MCP client factory with graceful fallback
|   ├── mcp_server/            ✅ NEW — standalone MCP server (Day 20 addition)
|   |   ├── __init__.py
|   |   ├── __main__.py
|   |   └── server.py          ✅ 10 MCP tools; stdio (Claude Desktop) + SSE
transport
|   ├── api/
|   |   ├── __init__.py
|   |   ├── middleware.py      ✅ slowapi limiter — Redis if available, memory://
fallback
|   |   └── app.py             ✅ FastAPI + /query + /query/stream + /history +
/agents/status
|   |                             + /ingest + /teams/webhook; get_graph at module
level
|   ├── teams/
|   |   ├── __init__.py
|   |   ├── models.py          ✅ TeamsActivity, TeamsUser, TeamsConversation
```

```
(Pydantic V2)
|   |   ├── cards.py          ✅ Adaptive Card builders
|   |   └── bot.py            ✅ Webhook router + HMAC verification + activity
routing
|   ├── config/
|   |   ├── __init__.py
|   |   └── settings.py       ✅ AppConfig + LLMConfig + RedisConfig +
DatabricksConfig
|   |                             + VectorDBConfig (Neon DATABASE_URL aware; all
default_factory)
|   |                             + DATA_PRODUCTS dict + get_config() singleton +
reset_config()
|   ├── ingestion/
|   |   ├── __init__.py
|   |   ├── loaders.py        ✅ Module-level LangChain loaders
(PDF/DOCX/PPTX/Excel/text)
|   |   ├── chunker.py        ✅ RecursiveCharacterTextSplitter(512/64) +
metadata tag
|   |   ├── embedder.py       ✅ OpenAI text-embedding-3-small, batch=100
(module-level import)
|   |   └── store.py          ✅ Neon-compatible psycopg2 + PGVector upsert +
SHA-256 dedup
|   ├── tools/
|   |   └── __init__.py
|   ├── ui/
|   |   ├── __init__.py
|   |   └── app.py            ✅ Streamlit chat UI + HITL panel + execution stats
|   └── __init__.py
├── scripts/
|   ├── init_pgvector.sql             ✅ Local/self-hosted pgvector DB setup
|   ├── init_neon.sql                 ✅ Neon-specific init (pgvector ext,
governance_rules,
|   |                                    audit_log, HNSW index comment)
|   └── trigger_ingest.sh             ✅ curl Airflow REST API → on_demand DAG
├── tests/
|   ├── conftest.py                   ✅ Shared fixtures
|   ├── test_day1.py                  ✅ Config + logging basics
|   ├── test_day14.py                 ✅ LiteLLM + Redis cache
|   ├── test_day15.py                 ✅ Retry + HITL
|   ├── test_day16.py                 ✅ FastAPI endpoints
|   ├── test_day17.py                 ✅ Teams bot + HMAC
|   ├── test_day18.py                 ✅ pgvector + MCP client
|   ├── test_day19.py                 ✅ Ingestion pipeline (patch.object for
psycopg2)
|   ├── test_day20_smoke.py           ✅ 15 end-to-end smoke tests
|   ├── test_day20_coverage.py        ✅ Coverage boosters — guardrails, cache,
retry, nodes
```

```
|   ├── test_day20_production.py        ✅ FastAPI, error handling, protocols
|   ├── test_mcp_server.py              ✅ MCP tool handlers (no transport
needed)
|   ├── test_services.py                ✅ Protocol conformance + factory
switching
|   ├── test_capacity_agent.py          ✅ Injection pattern
|   ├── test_information_agent.py       ✅ Injection pattern
|   ├── test_knowledge_agent.py         ✅ Injection pattern
|   ├── test_metadata_agent.py          ✅ Injection pattern
|   └── test_rule_agent.py              ✅ Rule CRUD + evaluate invariant
├── data/
|   └── memory.db                       ✅ SQLite placeholder (MemorySaver /
SqliteSaver in dev)
├── docs/
|   └── .gitkeep                        ← Airflow FileSensor watches this folder
├── .github/
|   └── workflows/
|       └── ci.yml                      ✅ GitHub Actions: lint (ruff) + pytest -
-cov + Docker build
├── claude_desktop_config_example.json  ✅ MCP server config snippet for Claude
Desktop
├── Dockerfile                          ✅ Two-stage build (builder + runtime),
python:3.12-slim
├── Dockerfile.airflow                  ✅ Extends apache/airflow:2.10, installs
project deps
├── compose.yml                         ✅ app + api + mcp_server + redis +
airflow services
|                                          NO local postgres — Neon used for all
PG
├── .env.example                        ✅ Full reference with all vars
├── pyproject.toml                      ✅ "integration"/"smoke" marks + coverage
fail_under=80
├── Makefile                            ✅ Common dev commands
└── requirements.txt                    ✅ All deps including psycopg2-binary,
langchain-postgres, mcp
```

Test suite result (Day 20): 240 passed, 0 failed, 84% coverage

---

AgentState (state.py — all fields)

python

```python
class AgentState(TypedDict, total=False):
    # Input
    query, thread_id, user_id, time_range, data_products

    # Routing (set by supervisor_node)
    intent, next_agents

    # Agent outputs (accumulated via operator.add — Annotated)
    agent_results, sources, anomalies, errors

    # Write actions
    auto_tickets, pending_action, approved

    # Final output
    final_summary, confidence

    # Memory
    conversation_history, user_preferences

    # Metadata
    execution_ms, query_id

    # Hook fields
    start_time, guardrail_passed, guardrail_reason

    # Sequential routing tracker
    _agents_ran: List[str]
```

`initial_state()` factory function builds a clean state for testing — accepts all fields as kwargs.

Graph Flow (Sequential Routing)

```
START
  → pre_hook          ← guardrails (length/SQL/injection/PII), start_time, query_id
      ↓ guardrail_passed=True          ↓ guardrail_passed=False
  → supervisor                         → post_hook → END
      → _agent_router (conditional)
          → first agent in next_agents[]
          → _after_first_agent (conditional loop through remaining agents)
              information_node  ← @cached_node(ttl=1800) + retry_agent_call()
              knowledge_node    ← @cached_node(ttl=7200) + retry_agent_call()
              metadata_node     ← @cached_node(ttl=3600) + retry_agent_call()
              capacity_node
              rule_node
```

```
          → auto_ticket    ← HITL gate
          → synthesizer    ← LiteLLM (Groq primary) + string fallback
          → post_hook      ← execution_ms
          → END

NOTE: Sequential routing avoids LangGraph InvalidUpdateError on non-Annotated
state keys. _agents_ran[] tracks which agents have run; _wrap_agent() appends.
_graph singleton cached in get_graph(); copilot_graph proxy delegates to it.
```

Intent → Agent Routing (routing.py)

python

```python
INTENT_AGENT_MAP = {
    "full_diagnostic":  ["information","knowledge","metadata","capacity"],
    "data_quality":     ["metadata","information"],
    "governance":       ["metadata","knowledge"],
    "incident_review":  ["capacity"],
    "knowledge_lookup": ["knowledge","metadata"],
    "metric_analysis":  ["information","knowledge"],
    "write_ticket":     ["capacity"],
    "write_metadata":   ["metadata"],
    "write_rule":       ["rule"],
    "unknown":          ["information","knowledge"],
}
```

IntentClassification Schema (intent.py)

python

```python
class IntentClassification(BaseModel):
    intent:        QueryIntent   # 10 enum values (QueryIntent str Enum)
    data_products: List[str]     # ["retention","bookings","cac","ltv"]
    confidence:    float         # 0.0-1.0
    reasoning:     str           # one-sentence rationale

# Chain: get_structured_llm(config, IntentClassification).invoke(prompt)
# Fallback: _keyword_fallback() when LLM unavailable
# KEYWORD_MAP covers all 9 non-unknown intents
```

LiteLLM Fallback Chain (llm_factory.py)

python

```
# Primary: Groq (llama-3.3-70b-versatile) via ChatGroq
# Fallback order (LLMConfig.fallback_models):
#   gpt-4o-mini → anthropic/claude-haiku-4-5 → gemini/gemini-1.5-flash
# No GROQ_API_KEY → _MockLLM stub (always returns string, never raises)
# API: get_llm(config, streaming=False) → BaseChatModel
#      get_structured_llm(config, schema) → LLM.with_structured_output(schema)
```

Guardrails (core/guardrails.py)

```
Check order (first match blocks or redacts):
  1. Length          min=3, max=2000 chars         → block
  2. Destructive SQL DROP/DELETE/TRUNCATE/ALTER     → block
  3. Prompt injection "ignore instructions", DAN    → block
  4. PII redaction   SSN, cards, emails, UK NI      → redact silently, allow

Returns GuardrailResult(passed, query, reason)
pre_hook sets state["guardrail_reason"] for downstream access.
```

Redis Cache (core/cache.py)

python

```
# Node TTLs:
@cached_node("information_agent", ttl=1800)   # 30 min — SQL results
@cached_node("knowledge_agent",   ttl=7200)   # 2 hrs  — RAG docs
@cached_node("metadata_agent",    ttl=3600)   # 1 hr   — Collibra metadata

# NOT cached: capacity_node, auto_ticket_node, synthesizer_node
# Cache key = SHA-256(query + data_products + time_range) via make_key()
# invalidate_pattern(client, pattern) → int (count of deleted keys)
# _redis_client and _in_memory fallback dict at module level
```

Rate Limiter (api/middleware.py)

python

```
# Probes Redis at import time (1-second timeout).
# Redis reachable → shared limits across ECS tasks.
# Fallback → memory:// per-process limits (dev/CI).
limiter      = Limiter(key_func=get_remote_address, storage_uri=_storage_uri)
user_limiter = Limiter(key_func=get_user_id,        storage_uri=_storage_uri)
```

Retry Logic (core/retry.py)

python

```
# Backoff: 1s → 2s → 4s (exponential, backoff_factor * 2^attempt)
@with_retry(max_retries=3, backoff_factor=1.0)
retry_agent_call(agent.execute, request, max_retries=3)
# Returns AgentResult(success=False) on final failure
# Applied to: information_node, knowledge_node, metadata_node
```

Human-in-the-Loop (nodes.py — auto_ticket_node)

```
Flow:
  1st call (approved=False):  anomalies found → set pending_action → skip tickets
  2nd call (approved=True):   create Jira tickets → clear pending_action

Critical HITL keywords: threshold, missing, below, risk, drop, fail

Streamlit: _render_hitl_panel() checks st.session_state.pending_hitl
Teams:     build_hitl_card() renders Approve/Reject Action.Submit buttons
```

FastAPI Endpoints (api/app.py)

```
GET  /health             → liveness probe (no rate limit)
POST /query              → JSON response, 20/min per IP
POST /query/stream       → SSE: start → result → done events, 20/min per IP
GET  /history/{thread}   → conversation history, 60/min
GET  /agents/status      → Redis + agents + daily token usage, 120/min
POST /teams/webhook      → Teams bot, 10/min per X-User-Id
GET  /teams/health       → Teams bot probe
POST /ingest             → multipart upload → triggers Airflow on_demand DAG,
10/min
```

```
_run_graph() helper invokes LangGraph; module-level get_graph import (patchable).
Run: uv run uvicorn src.api.app:app --reload --port 8000
```

MCP Server (src/mcp_server/server.py) ✅ NEW

```
Transports:
  stdio  (default) — Claude Desktop / local clients
  SSE    (TRANSPORT=sse, MCP_PORT=8002) — remote/web clients

10 MCP Tools:
  1.  governance_query        → full LangGraph pipeline
  2.  query_metrics           → Information Agent (Databricks / mock)
  3.  search_knowledge_base   → Knowledge Agent (pgvector RAG)
  4.  get_metadata            → Metadata Agent (Collibra)
  5.  manage_incidents        → Capacity Agent (Jira)
  6.  manage_rules            → Rule Agent (CRUD + evaluate)
  7.  get_system_status       → Redis + agents + daily token usage
  8.  invalidate_cache        → by agent name or "all"
  9.  approve_action          → re-run graph with approved=True (thread-scoped)
  10. ingest_document_url     → download URL → load→chunk→embed→upsert

Claude Desktop config: claude_desktop_config_example.json
Compose service: mcp_server (port 8002, TRANSPORT=sse)
```

Services Layer (services/)

```
Design: Interface protocol → Factory → Real or Mock implementation
Switching: ENABLE_MOCK=true → mock; ENABLE_MOCK=false → real with graceful
fallback

services/base.py    → 4 runtime_checkable Protocol classes:
                      IDataService, ITicketService, IMetadataService,
IVectorService
services/factory.py → get_data_service(), get_ticket_service(),
                      get_metadata_service(), get_vector_service()
```

Ingestion Pipeline (src/ingestion/)

python

```
# All imports at module level for patchability in tests
loaders.py  — LangChain loaders: PDF/DOCX/PPTX/Excel/text (load_document())
chunker.py  — RecursiveCharacterTextSplitter(512/64) + content_hash + product infere
embedder.py — OpenAIEmbeddings(text-embedding-3-small), batch=100
store.py    — Neon-compatible psycopg2 + PGVector upsert + SHA-256 dedup

Test pattern: patch.object(store_module.psycopg2, "connect", ...)
```

Airflow DAGs (compose.yml airflow services)

```
Note: DAG files not present in this zip; referenced in compose.yml and scripts/.
Expected structure in dags/ directory:
  file_ingestion_dag.py      schedule=None  (FileSensor on AIRFLOW_DOCS_PATH)
  on_demand_ingest_dag.py    schedule=None  (REST API / POST /ingest)
  collibra_sync_dag.py       schedule="0 6 * * *"  (daily 06:00 UTC)
  nightly_refresh_dag.py     schedule="0 2 * * *"  (nightly 02:00 UTC)

Airflow metadata DB: AIRFLOW_DB_URL (Neon direct URL recommended, not pooled)
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN = ${AIRFLOW_DB_URL:-${DATABASE_URL}}
```

Neon PostgreSQL Integration

```
Primary driver: DATABASE_URL environment variable (pooled pgBouncer URL)
Fallback: individual POSTGRES_HOST/PORT/USER/PASSWORD/DB/SSLMODE vars

VectorDBConfig.__post_init__() parses DATABASE_URL and overrides individual fields
checkpointer.py _pg_url() normalises scheme to postgresql:// for PostgresSaver
store.py _psycopg2_kwargs() parses DATABASE_URL for raw psycopg2 connections
sslmode=require enforced everywhere for Neon

One-time setup: psql "$DATABASE_URL" -f scripts/init_neon.sql
HNSW index: CREATE after first ingestion (commented in init_neon.sql)
```

Docker Compose Services

```
redis          → redis:7-alpine, port 6379, LRU eviction, health-checked
app            → Streamlit UI, port 8501
api            → FastAPI/uvicorn, port 8000
```

```
mcp_server     → MCP SSE server, port 8002
airflow-init   → DB migrate (one-shot)
airflow-webserver → Airflow API server, port 8080
airflow-scheduler → DAG scheduler

NO local postgres service — all PG connections go to Neon via DATABASE_URL
```

Environment Variables (key ones)

bash

```
# LLM
LLM_PROVIDER=groq
LLM_MODEL=llama-3.3-70b-versatile
GROQ_API_KEY=
ANTHROPIC_API_KEY=          # LiteLLM fallback
GEMINI_API_KEY=             # LiteLLM fallback
OPENAI_API_KEY=             # embeddings + GPT fallback

# App
ENABLE_MOCK=true            # false in prod
ENVIRONMENT=development     # production switches PostgresSaver
DEBUG=false
LOG_LEVEL=INFO

# Neon
DATABASE_URL=               # postgresql://user:pass@ep-xxx.../db?sslmode=require
AIRFLOW_DB_URL=             # separate direct URL for Airflow (optional)
POSTGRES_SSLMODE=require

# Redis
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_ENABLED=true

# MCP
TRANSPORT=stdio             # or "sse"
MCP_PORT=8002
USE_MCP=false               # true routes Collibra/Jira through MCP

# LangSmith
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=
LANGSMITH_PROJECT=gitcopilot

# Airflow
AIRFLOW_DOCS_PATH=./docs
AIRFLOW__CORE__FERNET_KEY=
```

Dev/Prod Config Switch Table

Variable	Dev	Prod
`ENABLE_MOCK`	`true`	`false`

Variable	Dev	Prod
`ENVIRONMENT`	`development`	`production`
`REDIS_ENABLED`	`false` (CI) / `true` (local)	`true`
`USE_MCP`	`false`	`true` (optional)
`DATABASE_URL`	unset (SQLite fallback)	Neon pooled URL
`MEMORY_BACKEND`	MemorySaver / SqliteSaver	PostgresSaver
`TRANSPORT`	`stdio`	`sse`

Complete Bug Fix Log (Days 1–20)

Days 1–19 fixes

All 35 bugs from Days 1–19 remain fixed (see prior context versions for full list).

Day 20 fixes

#	File	Bug	Fix
36	`src/graph/graph.py`	Parallel fan-out → `InvalidUpdateError` on non-Annotated keys	Sequential routing + `_wrap_agent()` + `_agent` tracker
37	`src/memory/checkpointer.py`	`langgraph.checkpoint.sqlite` not available	Fallback chain: SqliteSaver langgraph_checkpoint_sql MemorySaver
38	`src/api/app.py`	`get_graph` imported inside function, not patchable	Moved to module-level
39	`src/ingestion/store.py`	`psycopg2` imported inside function	Moved all imports to modu tests use `patch.object`
40	`src/ingestion/embedder.py`	`OpenAIEmbeddings` imported inside function	Moved to module level
41	`src/ingestion/loaders.py`	Loader classes imported inside function	Moved to module level
42	`tests/test_day20_coverage.py`	`_keyword_fallback("create rule for data quality")` matched wrong intent	Fixed to `"create rule fo retention"`

#	File	Bug	Fix
43	`tests/test_day20_production.py`	FastAPI `/query` test ran real graph	Patched `_run_graph` with result
44	`src/graph/state.py`	`_agents_ran` field missing from TypedDict	Added `_agents_ran: Lis`
45	`src/config/settings.py`	Local postgres hardcoded; Neon URL not handled	`DATABASE_URL` parsing in `VectorDBConfig.__post_` + `psycopg2_dsn` property
46	`compose.yml`	Local postgres service present; no MCP server service	Removed postgres, added `mcp_server` service (TRANSPORT=sse, port 8C
47	`scripts/init_neon.sql`	No Neon-specific init script	Added `init_neon.sql` w pgvector ext, governance_r audit_log, HNSW index cor

Current Status

Day 20 Complete — 240 passed, 0 failed, 84% coverage

All Days 1–20 Complete + MCP Server:

Day / Feature	Focus	Status
Days 1–4	Project setup, config, logging, base_agent	✅
Day 6	InformationAgent	✅
Day 7	KnowledgeAgent	✅
Day 8	MetadataAgent	✅
Day 9	CapacityAgent	✅
Day 10	RuleAgent	✅
Days 11–12	Write capabilities, LangGraph + memory + Streamlit	✅
Day 13	Intent + synthesizer + hooks + guardrails + LangSmith	✅
Day 14	LiteLLM fallback + Redis cache + @cached_node	✅
Day 15	retry.py + HITL + Streamlit HITL panel	✅

Day / Feature	Focus	Status
Day 16	FastAPI + SSE + rate limiting + llm_guard	✅
Day 17	Teams bot + Adaptive Cards + HMAC + HITL buttons	✅
Day 18	pgvector + NullVectorStore + MCP client + VectorDBConfig	✅
Day 19	Docker + CI/CD + Airflow RAG ingestion pipeline	✅
Day 20	Smoke tests + 84% coverage + production cleanup	✅
Bonus	**MCP Server (10 tools, stdio + SSE)**	✅
Bonus	**Neon PostgreSQL integration**	✅

8 Additional Features

Feature	Status
Persistent memory (PostgresSaver / SqliteSaver)	✅
Node caching (Redis + in-memory fallback)	✅
Pre/post hooks	✅
Error handling & retries	✅
Human-in-the-Loop	✅
Logging & monitoring (LangSmith)	✅
Guardrails (length, SQL, injection, PII)	✅
MCP integration (client + standalone server)	✅
RAG ingestion pipeline (Neon-compatible)	✅

How to Run

```bash
```

```
# Install dependencies
uv pip install -r requirements.txt

# Run all tests with coverage
ENABLE_MOCK=true REDIS_ENABLED=false \
  uv run pytest tests/ --cov=src --cov-fail-under=80

# Run only smoke tests
uv run pytest tests/test_day20_smoke.py -v

# Run MCP server tests
uv run pytest tests/test_mcp_server.py -v

# Run Streamlit UI
uv run streamlit run src/ui/app.py

# Run FastAPI server
uv run uvicorn src.api.app:app --reload --port 8000

# Run MCP server (stdio — Claude Desktop)
python -m src.mcp_server.server

# Run MCP server (SSE — remote clients)
TRANSPORT=sse MCP_PORT=8002 python -m src.mcp_server.server

# Run with Docker (full stack — requires DATABASE_URL in .env)
docker compose up

# Initialise Neon DB (one-time)
psql "$DATABASE_URL" -f scripts/init_neon.sql

# Trigger document ingestion
bash scripts/trigger_ingest.sh path/to/doc.pdf
curl -X POST http://localhost:8000/ingest -F "file=@doc.pdf"
```

Key Architectural Decisions & Notes

1. **Sequential routing (not parallel)** — avoids `InvalidUpdateError` on non-Annotated AgentState keys; `_agents_ran` list tracks progress.
2. **Neon replaces local Postgres** — single `DATABASE_URL` drives all connections; `VectorDBConfig` and `checkpointer.py` both parse it.
3. **MCP Server is standalone** — runs independently of FastAPI; shares the same agent/graph code via `src/` imports; supports both Claude Desktop (stdio) and remote (SSE) clients.

4. **All module-level imports —** psycopg2, OpenAIEmbeddings, LangChain loaders must be at module level to be patchable with `patch.object()` in tests.
5. **Singleton graph —** `_graph` singleton in `graph.py`; `copilot_graph` proxy for backward compat; tests patch `get_graph` or `_run_graph` at module level.
6. **Config singleton —** `get_config()` + `reset_config()` pattern; all fields use `default_factory` for env var reads at instantiation time.